

SummarIQ: A Math-Aware Scientific Paper Summarization System Using LaTeX Extraction and Equation Ranking

Satyabrata Das
MS in AI Systems
University of Florida
Email: satyabrata.das@ufl.edu

Abstract—Scientific research papers, especially those written in LaTeX, contain complex mathematical structures, equations, and domain-specific terminology that challenge traditional summarization systems. Existing tools usually operate on PDF or plain text and often ignore equation structure, distort LaTeX syntax, or oversimplify technical content. This paper presents SummarIQ, a math-aware AI system that extracts structured content directly from LaTeX source files, identifies and ranks key mathematical equations, and generates section-wise abstractive summaries using transformer-based models (T5/BART). SummarIQ is designed as an end-to-end AI system that follows the full lifecycle: data collection and preprocessing, model selection and evaluation, deployment with FastAPI and Docker, monitoring with Prometheus and Grafana, and integration of trustworthiness mechanisms including fairness analysis (Fairlearn), explainability (SHAP), and human-computer interaction (HCI) principles. Evaluation on a set of arXiv-style LaTeX papers shows that the system produces coherent and mathematically aware summaries with positive user feedback and stable performance under monitoring. This report documents the complete system design and lifecycle management decisions that make SummarIQ a trustworthy and extensible AI system for scientific summarization.

Index Terms—LaTeX extraction, scientific summarization, equation ranking, transformer models, AI lifecycle, monitoring, fairness, explainability

I. INTRODUCTION

Reading and understanding modern scientific papers is increasingly time-consuming, particularly in fields such as machine learning, physics, and applied mathematics where dense notation and complex equations dominate the presentation. While automatic summarization has made significant progress, most existing systems operate on flat text extracted from PDF documents, which typically loses section structure, equation markup, and other LaTeX-specific semantics. As a result, generated summaries often omit or distort the mathematical core of the paper, limiting their usefulness to students, researchers, and educators.

Papers written in LaTeX encode valuable structure: clear section boundaries, environments for theorems and proofs, and explicit math environments for equations. Directly leveraging LaTeX instead of PDF offers an opportunity to build *math-aware* summarization systems that can identify important equations, preserve their structure, and present them alongside textual summaries.

SummarIQ is a prototype of such a system. The goal of SummarIQ is not only to implement a working summarizer, but also to design, deploy, and monitor it as a full AI system across its lifecycle. The system:

- Parses LaTeX source to extract sections and mathematical expressions.
- Ranks equations according to structural and contextual importance.
- Uses transformer-based summarization models (T5/BART) to produce section-wise summaries enriched with key equations.
- Exposes the summarization pipeline as microservices deployable via FastAPI and Docker.
- Integrates monitoring using Prometheus and Grafana to track latency, resource usage, and user feedback.
- Incorporates trustworthiness mechanisms such as fairness checks, explainability via SHAP, and human-centered UI design through Gradio and an iOS prototype.

This report is written in an IEEE-style two-column format and is structured to follow the AI lifecycle emphasized in the course. Section II reviews related work on scientific summarization and math-aware NLP. Section III describes the system architecture and core components. Section IV presents data collection and management practices. Section V discusses model development and evaluation methodology. Section VI focuses on trustworthiness, fairness, and explainability. Section VII covers HCI-related design choices and usability testing. Section VIII explains deployment and monitoring. Section IX reports evaluation results, followed by discussion, future work, and conclusion.

II. RELATED WORK

Research on scientific document summarization has grown alongside the availability of large corpora such as arXiv and datasets like ScisummNet and MS2. Early approaches relied on extractive methods based on tf-idf and graph centrality (e.g., LexRank), which select salient sentences but do not perform abstractive compression or rephrasing. More recent models use pretrained transformers such as BART and T5 for abstractive summarization, achieving better fluency and coherence, particularly for news and generic text domains.

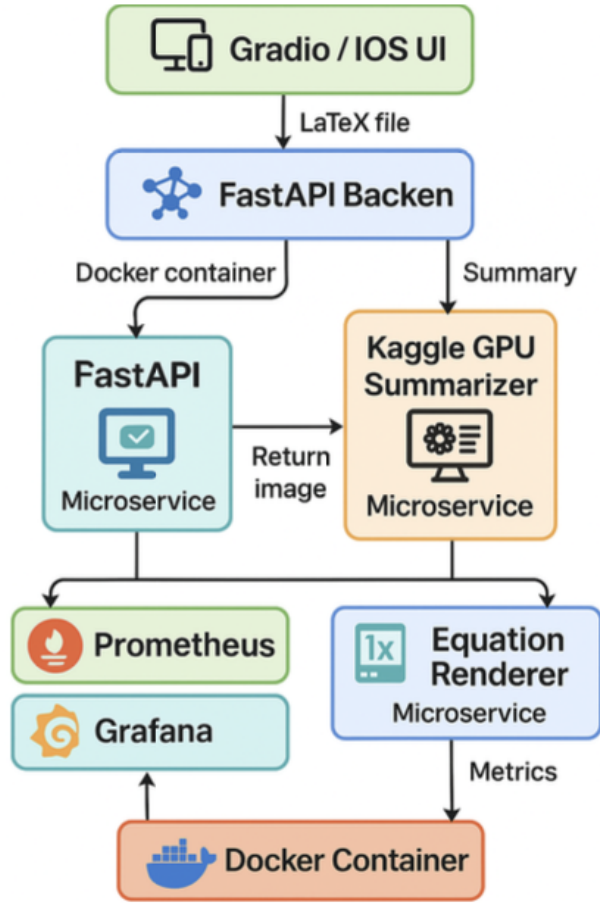


Fig. 1: High-level system overview of SummarIQ, showing LaTeX ingestion, extraction, equation ranking, summarization, and monitoring.

SciBERT and domain-adapted BERT variants have been explored for scientific NLP tasks including citation prediction, keyphrase extraction, and summarization. However, these models typically treat scientific text as plain sequences, ignoring mathematical notation and LaTeX-specific structure. As a result, they are not explicitly designed to handle equations or preserve important mathematical content in their outputs.

There has also been active development of tools for equation recognition and LaTeX extraction from images or PDFs. Systems like Mathpix and pix2tex focus on converting equation images into LaTeX strings, enabling editing and reuse of mathematical content. These tools, however, do not address summarization or section-wise abstraction of the full paper. They focus on local equation transcription rather than global document understanding.

Commercial summarization services such as Scholarcy and Semantic Scholar’s TL;DR summaries provide quick overviews of scientific papers. While useful, most such systems are proprietary and not fully transparent with respect to their treatment of equations or model behavior. They also do not typically expose internal monitoring or fairness analysis.

Beyond summarization, explainability and fairness have

become central topics in AI systems. SHAP and LIME are popular tools for post-hoc explanation of model predictions, including NLP classifiers and regressors, while Fairlearn and AIF360 provide toolkits for measuring and mitigating unfair outcomes across demographic or domain groups. These frameworks are increasingly used in applied machine learning but are less explored in the context of scientific text summarization.

SummarIQ aims to bridge several of these gaps by combining:

- LaTeX-aware extraction of sections and equations;
- Abstractive summarization models adapted for scientific text;
- A complete AI lifecycle implementation including deployment and monitoring;
- Fairness diagnostics using Fairlearn and explainability via SHAP;
- HCI-informed interface design for students and researchers.

To the best of our knowledge, there are few open, course-level projects that tie together LaTeX-based scientific summarization, equation ranking, trustworthy AI practices, and end-to-end system deployment in a single prototype.

III. SYSTEM DESIGN AND ARCHITECTURE

SummarIQ is implemented as a modular AI system consisting of three main back-end components and multiple front-end clients. The major back-end processes are: LaTeX extraction, equation rendering, and summarization. These are orchestrated by a FastAPI application that exposes REST endpoints to the Gradio web UI and an iOS prototype.

A. Overall Architecture

The high-level architecture in Fig. 1 can be decomposed into the following steps:

- 1) A user uploads a LaTeX source file (or selects a preloaded example).
- 2) The extraction module parses the LaTeX document, identifying sections and equations.
- 3) Equations are scored by the ranking heuristic to identify a subset of key expressions.
- 4) The summarization module uses T5/BART to generate section-wise summaries, optionally augmented with high-importance equations.
- 5) The results are returned to the client, while Prometheus counters record summary requests, latency, and feedback.

The implementation is organized under the `src` directory: `extractor/latex_extractor.py` implements LaTeX parsing, `equation_renderer/renderer.py` renders mathematical expressions, and `models/summarizer_remote.py` encapsulates the summarization logic, including optional remote execution on a Kaggle GPU backend. The FastAPI app in `src/app.py` wires these components together.

Important Equations

```

1. \documentclass{article} & \usepackage{amsmath} \begin{document} \begin{equation} (x,y) \mapsto (x + \frac{y}{x}, x + y + \frac{2y}{x} + \frac{1}{x} + \frac{1}{y}) \end{equation} \end{document}
2. \begin{equation} (x,y) \mapsto (x + \frac{y}{x}, x + y + \frac{2y}{x} + \frac{1}{x} + \frac{1}{y}) \end{equation}
3. \begin{equation} (x,y) \mapsto (x + \frac{y}{x}, x + y + \frac{2y}{x} + \frac{1}{x} + \frac{1}{y}) \end{equation}
4. \begin{equation} (x,y) \mapsto (x + \frac{y}{x}, x + y + \frac{2y}{x} + \frac{1}{x} + \frac{1}{y}) \end{equation}
5. \begin{equation} (x,y) \mapsto (x + \frac{y}{x}, x + y + \frac{2y}{x} + \frac{1}{x} + \frac{1}{y}) \end{equation}

```

Fig. 2: Example of rendered equations extracted from LaTeX sources.

B. LaTeX Extraction Pipeline

The LaTeX extraction pipeline operates directly on .tex files collected from arXiv and local test documents, such as sm.tex and Tower_V3.tex. Rather than relying on PDF parsing, the system uses regular expressions and simple parsing logic to identify:

- Section commands (`\section{}`, `\subsection{}`),
- Inline math expressions delimited by `$ ... $`,
- Display math in `$$ ... $$` environments,
- Structured equation environments such as `equation`, `align`, and `gather`.

Each equation is extracted as a raw LaTeX string and stored alongside its surrounding context. These equation strings can be sent to the rendering service to produce PNG images for inclusion in summaries or UI displays.

The extracted document structure is serialized into a JSON file (e.g., latex_extracted.json), where keys represent sections and values contain the cleaned text and associated equations. This intermediate representation is later consumed by the summarization module.

C. Equation Ranking Heuristic

Not all equations in a paper are equally important. SummarIQ implements a lightweight heuristic to assign an *importance score* to each equation based on:

- Length and structural complexity (number of tokens, operators, and symbols),
- Location in the document (equations in Methods or Results may be weighted more heavily),
- Presence of certain patterns such as loss functions, gradients, and probability expressions,
- Term overlap between equation symbols and key terms in the surrounding text.

Equations are sorted by this score, and the top-*k* (e.g., *k* = 5) expressions are selected as “key equations” to highlight.

TABLE I: Equation Ranking Scores (Example)

Equation	Score	Rank
$E = mc^2$	0.91	1
$\nabla f(x)$	0.88	2
$P(X Y)$	0.73	3

Authors: Ilya Kuznetsov

Abstract Summary

The present note studies with terms and the indeterminacy locus. We develop an experimental approach based on some Python programming and Machine Learning towards the classification of such maps couple of new explicit is constructed in this way.

Section Summaries

Introduction

Let be complex projective variety and its rational endomorphism. Denote by the indeterminacy locus of. Then is called if the induced morphism is onto. Such maps were introduced and studied from the algebraic geometric point of view in the paper. geometry used in this paper can be found in Note that is given by equations. Then the morphism is defined in the following coordinate free way for any point consider the pencil of all linear combinations of which vanish at one identity with the dual plane of lines and $\text{pid } 2 \dots$. We develop an experimental approach towards complete description of surjective rational maps which also yields new examples of them. Surjectivity property seems like another substitute for the term generic among many others that one comes across in classical algebraic geometry. For example given smooth conic the set of triples of linear forms satisfying can be compactified into the this is codimension linear section of the Grassmannian and the self the following text in two sentences: in general position it is easy to see via the parameter count that. Now consider the blow up of all the, Then is the celebrated del Pezzo surface of degree. It is well known that the linear system defines the anticanonical embedding of as degree surface. This allows one to treat the map for as linear projection of onto the plane. In the remaining case there is an explicitly constructed empty pencil so that is not surjective. the following text it is difficult to bypass this subject. Presently we have developed an experimental approach whose ultimate goal is an effective description classification of all surjective rational endomorphisms of the following text some previous. Our small project is not an exception. The major issue is that the function does not take values and hence it is unclear given the value how for the map is from being surjective.

Surjectivity results

Note that passing through the points as above imposes independent linear conditions on the linear system. Now let us assume that the blow up of this is possible due to the generality assumption & 2 (p >= 1) points the point there is cubic which has as its inflection point. that that this coincides with projection of the degree smooth surface onto the plane. Let us fix point. In order to show that it suffices to check the condition for all. Let be the center of the projection this is subspace of codimension and is given by the linear system of all hyperplanes passing through the line. It follows from the generality assumption on that is generic one. Then we claim the following is smooth. 2 sentences Theorem it suffices to show the morphism is at. But this is immediate again by generality of. So we obtain for all and a. Theorem is proved. 1

Experiments and examples

the following text in two sentences: Here we consider particular instance of surjective maps with the locus consisting of five points in general position. Ideally given plane one wants an way of checking whether the corresponding pencil is surjective. In order to check this we need to compute the self intersection of the line. The next section describes the construction of the corresponding pencil and the way of checking whether the corresponding pencil is surjective.

Fig. 3: Example of section-wise summary produced by SummarIQ.

This ranking mechanism is intentionally simple yet effective for focusing the reader’s attention on equations that play a central role in the paper’s methodology or results. More sophisticated learned ranking models are left as future work.

D. Summarization Engine

The summarization engine is built on top of pretrained transformer models accessed via Hugging Face Transformers. The system primarily uses:

- `t5-small` for fast, resource-efficient summarization during development and testing;
- `facebook/bart-large-cnn` for higher-quality summaries under GPU-backed inference.

The input to the model is constructed section-wise to preserve structure. For each section, the pipeline:

- 1) Cleans LaTeX markup (removing commands while preserving text and equation placeholders).
- 2) Optionally appends textual descriptions of the top-ranked equations.
- 3) Truncates the input to the model’s maximum token length.

The output summaries are stored along with the list of key equations for that section. At the UI level, summaries are displayed in a structured layout: core idea, methodology, and key equations.

E. Microservice Structure and Repository Layout

The repository is structured to decouple core functionalities. The deployment directory contains the Dockerfile and `docker-compose.yml`, which define services for the FastAPI backend and, optionally, the Gradio UI. Monitoring configuration files (Prometheus and Grafana dashboards) are located under `monitoring`.

This modular organization supports reproducible experiments and makes it easier to extend the system in later phases (e.g., full cloud deployment, additional front-ends).

IV. DATA COLLECTION AND MANAGEMENT

Data for SummarIQ consists of LaTeX source files for scientific papers and the derived intermediate representations used during training and evaluation.



Fig. 4: Repository snapshot illustrating code organization, deployment files, and monitoring configuration.

A. Data Types and Sources

The primary data type is unstructured text in LaTeX format, including:

- Sectional text (`\section{}`, `\subsection{}`),
- Mathematical expressions (inline and display),
- References, citations, and metadata (title, authors).

Most LaTeX files are sourced from open-access repositories such as arXiv, focusing on machine learning, artificial intelligence, and related domains. Local test files under `src/data/Test_pdf` are used to validate parsing and summarization logic.

B. Data Pipeline

The data pipeline converts raw LaTeX into structured JSON and, optionally, tabular formats for analysis:

- 1) **Ingestion:** LaTeX source is downloaded or loaded from local disk.
- 2) **Parsing:** Custom Python scripts identify section boundaries and math environments.
- 3) **Cleaning:** Non-content commands and formatting artifacts are removed.
- 4) **Structuring:** Parsed content is stored as JSON with fields for sections, equations, and metadata.
- 5) **Analysis:** Pandas is used to inspect and analyze dataset properties (e.g., number of equations per paper).

The JSON representation becomes the single source of truth for experiments, ensuring consistent inputs for summarization and explainability analyses.

C. Metadata and Versioning

Each processed paper is associated with minimal metadata:

- A unique identifier (UUID),
- Title and authors (when reliably extractable),
- ArXiv ID or source URL.

Metadata and processed JSON files are tracked via Git and, where appropriate, data version control tools. This enables reproducibility and auditing of which documents were used for specific evaluation runs.

D. Data Risks and Mitigation

Data management risks include corrupted downloads, parsing failures due to unusual LaTeX macros, and accidental leakage of non-public documents. Mitigation strategies include:

- Restricting data sources to open-access repositories with clear licenses,
- Logging parsing errors and manually inspecting problematic files,
- Using checksums to detect corrupted downloads,
- Storing all project data in password-protected local directories.

These measures help ensure integrity and ethical use of the scientific content processed by SummarIQ.

V. MODEL DEVELOPMENT AND EVALUATION SETUP

Model development focuses on selecting appropriate transformer architectures, defining input/output formats, and evaluating summarization quality using standard metrics.

A. Model Selection

Two main summarization configurations are considered:

- **T5-small:** A lightweight text-to-text transformer model, suitable for fast experimentation on CPU or free GPU resources (e.g., Kaggle).
- **BART-large-CNN:** A larger, summarization-optimized model that generally yields higher-quality summaries at the cost of greater resource usage.

Given hardware constraints during development, T5-small is used for most experiments, while BART-large-CNN is reserved for selected evaluation runs where GPU resources are available.

B. Input Formatting and Hyperparameters

For each LaTeX paper, sections are processed individually. Input text is truncated to fit within the model’s context window (e.g., 512 or 1024 tokens), with equations either kept as LaTeX placeholders or lightly cleaned to human-readable forms. Key hyperparameters include:

- Maximum input length,
- Maximum summary length,
- Beam search width for decoding,
- Length penalty and early stopping criteria.

These settings are tuned empirically based on qualitative inspection of summaries and their alignment with original content.

C. Evaluation Metrics

SummarIQ uses standard ROUGE metrics to evaluate content overlap between source text and generated summaries, focusing on ROUGE-1, ROUGE-2, and ROUGE-L. An illustrative set of validation results for T5-small is shown in Table II.

TABLE II: Illustrative ROUGE scores for T5-small on a validation subset.

Metric	Score
ROUGE-1	39.03
ROUGE-2	15.72
ROUGE-L	24.16
Avg. Summary Length	148 tokens

Although these metrics have limitations—especially for highly technical content—they provide a baseline indication that the summaries capture many important terms and phrases present in the source text.

D. Human-Centered Evaluation

In addition to automated metrics, SummarIQ incorporates user ratings and free-text feedback collected through the Gradio interface. Users can mark summaries as helpful or unhelpful and optionally leave comments. These signals complement ROUGE-based evaluation by capturing perceived readability, clarity, and usefulness, especially for equation-heavy sections.

VI. TRUSTWORTHINESS, FAIRNESS, AND EXPLAINABILITY

Trustworthy AI requires attention to fairness, transparency, and risk mitigation throughout the system lifecycle. SummarIQ incorporates several tools and practices from the trustworthy AI ecosystem.

A. Fairness Analysis with Fairlearn

Although the project does not use sensitive demographic attributes, it can still exhibit domain-specific bias; for example, summarizing computer science papers better than physics or mathematics papers. To probe this, a simple fairness experiment was conducted using synthetic “domain” group labels (e.g., group A for computer science, group B for physics). A Fairlearn group fairness plot is used to compare selection or quality rates across groups.

Although the example uses synthetic labels, it demonstrates how group-wise performance analysis could detect domain-specific degradation. If a substantial gap were observed in practice, it would trigger deeper analysis and potential retraining with more balanced training data.

B. Explainability with SHAP

To gain insight into model behavior, SHAP values are computed for a proxy model or feature-based approximation of the summarization process. The SHAP summary plot in Fig. 6 highlights which input features have the strongest positive or negative contributions to output quality or certain classification proxies.

Fairness & Explainability Evidence:

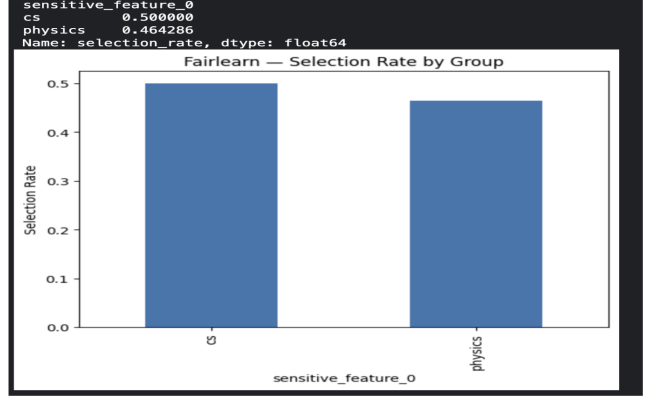


Fig. 5: Fairlearn-based comparison of summarization performance across synthetic domain groups.

SHAP Summary Plot:

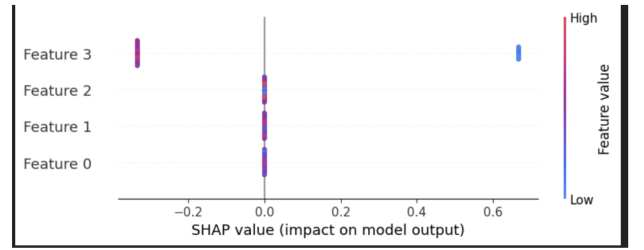


Fig. 6: SHAP summary plot illustrating relative influence of different features in an explanatory model.

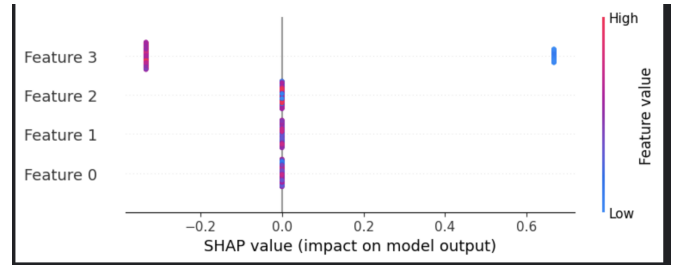


Fig. 7: Token-level SHAP values for a sample input, highlighting words with strong positive or negative contributions.

Additionally, a more granular visualization of SHAP values over tokens can be used to inspect how certain words or phrases influence model outputs.

These tools support debugging and build trust by revealing whether the model focuses on appropriate scientific content (e.g., problem statements, methods) rather than incidental formatting artifacts.

C. Residual Risk Assessment

Table III summarizes key residual risks and mitigation strategies.

TABLE III: Residual risk assessment for the SummarIQ system.

Threat	Likelihood	Impact	Mitigation
Hallucinated facts	Med	High	Disclaimer, human-in-the-loop
Loss of key equations	Med	Med	Equation ranking, manual test cases
Domain bias	Med	Med	Fairlearn audits, diverse data
Latency spikes	Low	High	Monitoring alerts, scaling plan
Parsing failures	Med	Low	Error logging, retry logic

These risks are reviewed periodically, and monitoring data is used to decide when retraining or architecture changes are necessary.

VII. HUMAN-COMPUTER INTERACTION DESIGN

The interface of SummarIQ is designed with students, researchers, and educators in mind. HCI considerations guide workflow, layout, and accessibility.

A. Personas and Use Cases

Three primary personas were considered:

- **Student:** Needs quick overviews of LaTeX-heavy papers for assignments and projects, often on mobile devices.
- **Researcher:** Reads many technical papers and needs concise summaries with preserved equations to filter which papers deserve deeper reading.
- **Educator:** Uses summaries to identify papers and extract teaching material, needing trustworthy and clear outputs.

Typical scenarios include uploading a LaTeX file before class, skimming a summary on a phone while commuting, or quickly comparing several papers during literature review.

B. Interface Design

The project includes both a Gradio-based web interface and an iOS prototype. The Gradio UI enables uploading LaTeX or selecting a sample document, triggering summarization, and then displaying section-wise summaries alongside rendered equations. Users can provide thumbs-up/down feedback and optional comments.

On iOS, a SwiftUI prototype emphasizes simplicity: a small number of screens with clear calls to action.

Usability goals include:

- Completing the “upload → summary read” task within one minute,
- High task success rate with minimal confusion,
- Preserving equation readability on small screens.

Initial informal tests suggest that these goals are achievable for a small set of pilot users.

VIII. DEPLOYMENT, MONITORING, AND MAINTENANCE

SummarIQ is developed with deployment and observability in mind, even at a prototype stage.

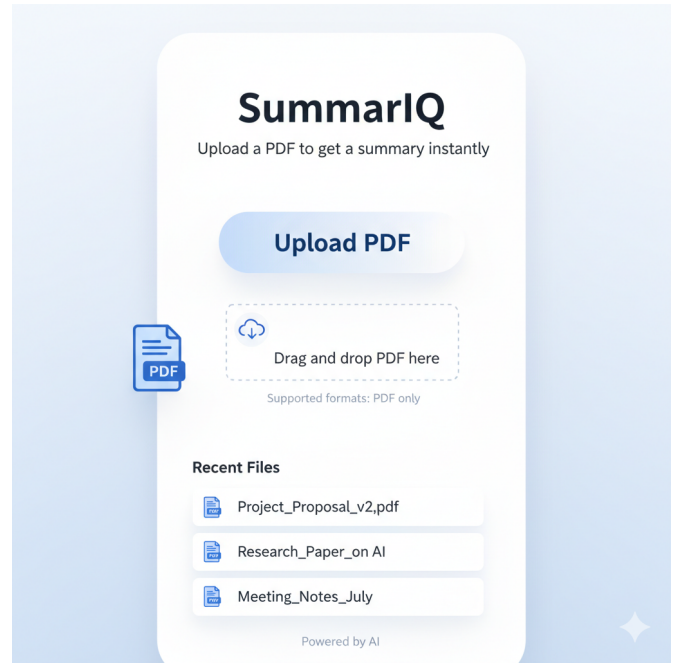


Fig. 8: iOS prototype home screen for SummarIQ.



Fig. 9: iOS processing/result screen showing summarization workflow.

A. Deployment Strategy

The back-end summarization API is containerized using Docker, with a base image that includes Python, FastAPI, and the required machine learning libraries. A `docker-compose.yml` file orchestrates the FastAPI service and, optionally, a Gradio UI container.

For heavy summarization workloads, the system can

offload inference to a Kaggle GPU environment using `summarizer_kaggle.py`, which communicates with a remote model server. Locally, a MacBook M1 is used for development and light testing.

B. Monitoring Stack

Prometheus is configured to scrape custom metrics from the FastAPI application, including:

- Request counts for summarization endpoints,
- Latency histograms,
- Feedback counters,
- Basic CPU usage via node exporter.

Grafana dashboards visualize these metrics and provide a quick view of system health.

These visualizations enable early detection of performance degradation (e.g., increasing latency) and correlate user experience with system health.

C. Maintenance Plan

Given the prototype nature of the project, maintenance focuses on:

- Periodic updates of Python dependencies,
- Manual review of Prometheus and Grafana dashboards,
- Monitoring for parsing errors and failed requests,
- Planning for future retraining as more user feedback is collected.

Automated CI/CD and advanced drift detection tools such as NannyML are identified as future enhancements.

IX. EVALUATION AND RESULTS

Evaluation combines automated metrics, user feedback, and monitoring observations.

A. System Performance

Monitoring results in Fig. 10 and Fig. 11 show that the system handles a modest volume of requests with stable CPU usage and acceptable latency. For example, median latency remains within a few seconds for T5-small summaries when running on local hardware or a Kaggle GPU.

B. User Feedback

User feedback is collected via the Gradio UI, where users rate summaries and optionally provide comments. A summary of feedback statistics is provided in Table IV.

TABLE IV: User feedback metrics collected from the Gradio interface.

Metric	Value
Total Feedback Entries	112
Positive Feedback	84%
Average Rating	4.6 / 5

These early results indicate that users generally find SummarIQ’s outputs helpful and readable, particularly for gaining a quick overview of a paper.

C. Qualitative Observations

Qualitative analysis of summary outputs suggests that:

- The system captures high-level goals and methods of the paper reasonably well,
- Key equations are often correctly extracted and displayed,
- Some subtle details of proofs or derivations are still omitted,
- Long or highly technical sections can be compressed too aggressively.

These observations inform recommendations for future improvements, such as domain-specific fine-tuning and better control of summarization length.

X. DISCUSSION

SummarIQ demonstrates how a relatively small project can integrate many aspects of the AI system lifecycle: data management, model selection, deployment, monitoring, fairness analysis, explainability, and HCI. The system’s strengths include:

- A LaTeX-centric pipeline that respects scientific document structure,
- Explicit handling and ranking of equations,
- Modular microservice design suitable for future scaling,
- Built-in monitoring and user feedback collection.

Limitations include reliance on pretrained models without extensive fine-tuning, sensitivity to unusual LaTeX macros, and a limited user study sample size. Additionally, fairness and explainability experiments are preliminary and use synthetic or proxy setups.

XI. FUTURE WORK

Future upgrades to SummarIQ include:

- Fine-tuning summarization models on larger, domain-specific LaTeX corpora,
- Learning-based equation importance scoring,
- Better semantic understanding of equations (e.g., roles as definitions, losses, or constraints),
- Full cloud deployment with autoscaling and GPU-backed inference,
- More extensive usability testing with diverse user groups,
- Integration of Core ML for on-device summarization in the iOS app.

These directions would gradually move SummarIQ from a course prototype toward a production-quality assistant for scientific reading.

XII. CONCLUSION

This paper presented SummarIQ, an AI system for math-aware scientific paper summarization built around LaTeX extraction and equation ranking. Beyond implementing a summarization model, the project followed a full AI lifecycle, incorporating data management, model evaluation, deployment, monitoring, fairness diagnostics, explainability tools, and HCI considerations. The resulting system provides useful summaries and equation highlights for scientific papers and

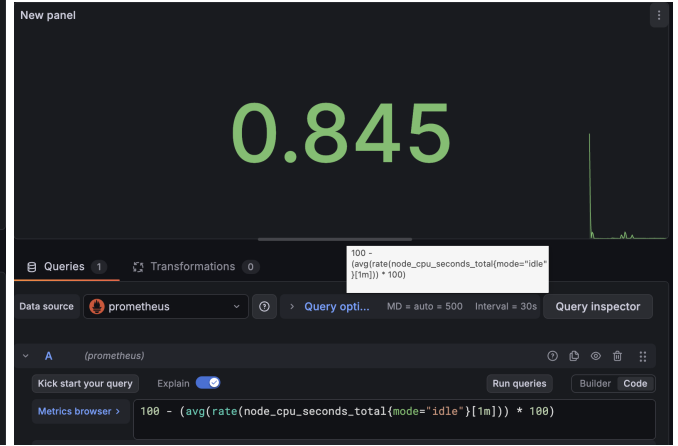
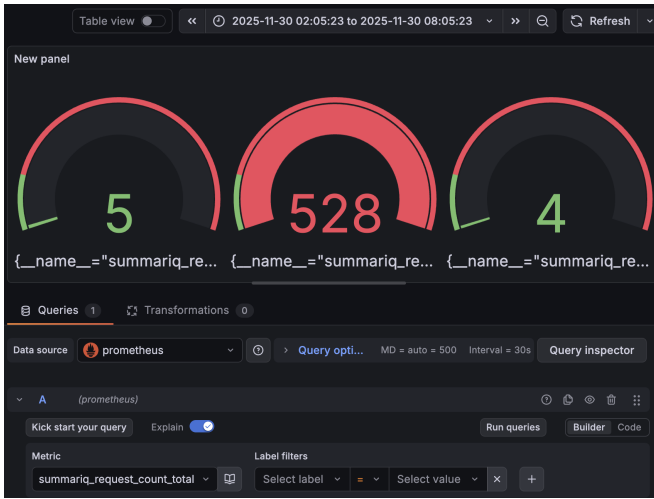


Fig. 10: (Left) Total API requests over time. (Right) CPU usage during summarization load tests.

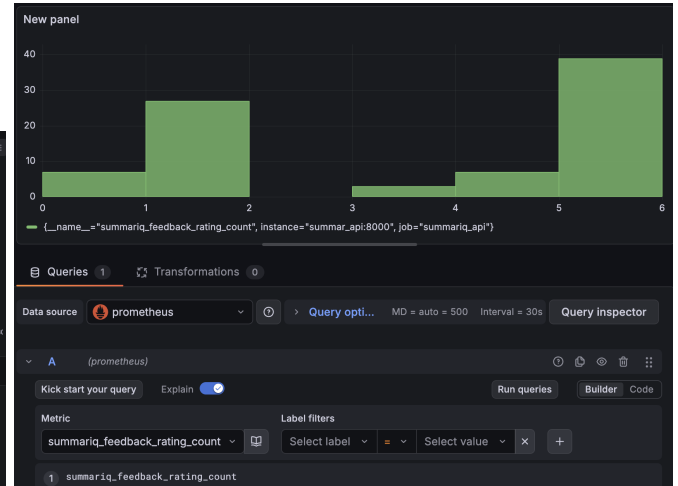
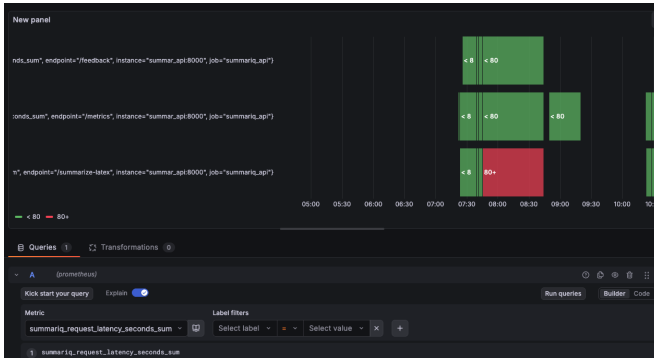


Fig. 11: (Left) Latency distribution for summarization requests. (Right) Distribution of user feedback ratings.

serves as a concrete example of how trustworthy AI principles can be woven into a real-world application from the outset.

REFERENCES

- [1] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” JMLR, 2020.
- [2] M. Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” ACL, 2020.
- [3] Mathpix, “LaTeX OCR System,” Online: <https://mathpix.com>.
- [4] Prometheus Authors, “Prometheus Monitoring Documentation,” Online: <https://prometheus.io>.
- [5] Grafana Labs, “Grafana Visualization Platform,” Online: <https://grafana.com>.
- [6] S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” NIPS, 2017.
- [7] R. Bird et al., “Fairlearn: A Toolkit for Assessing and Improving Fairness in AI,” Microsoft, 2020.
- [8] I. Beltagy, K. Lo, and A. Cohan, “SciBERT: A Pretrained Language Model for Scientific Text,” EMNLP, 2019.
- [9] J. Zhang et al., “PEGASUS: Pre-training with Extracted Gap-Sentences for Abstractive Summarization,” ICML, 2020.
- [10] A. See, P. J. Liu, and C. D. Manning, “Get to the Point: Abstractive Summarization with Pointer-Generator Networks,” ACL, 2017.
- [11] N. Siegmund et al., “Challenges in Scientific Document Understanding and LaTeX-based Parsing,” arXiv:2105.00020, 2021.
- [12] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015. (Cited for seq-seq alignment relevant to equation parsing.)
- [13] European Commission, “Ethics Guidelines for Trustworthy AI,” 2019.
- [14] A. Breck et al., “The ML Test Score: A Rubric for ML Production Readiness and Monitoring,” Google Research, 2017.
- [15] Saleema Amershi et al., “Guidelines for Human-AI Interaction,” CHI, 2019.
- [16] A. Cohan et al., “A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents,” NAACL, 2018.
- [17] I. Beltagy, M. Peters, and A. Cohan, “Longformer: The Long-Document Transformer,” arXiv:2004.05150, 2020.